

Search
Home
GitHub
CLASSES
ArticlesService
ErrorBoundary
EVENTS
SIGINT
unhandledRejection
NAMESPACES
apiConfig
contactoEmergenciaService
diarioService
GLOBAL
404_Error_Handler
ACTIVIDADES
API_URL
ASPECTOS_COGNITIVOS
ActionCard
ActivitySelector
App
Articulos
AuthButtons
AuthLayout
Button
CORS_Configuration
Card
Carousel
Checkbox
CircularProgress
CognitionSelector
Diario
DiaryBanner
DiaryEditor
DiaryEntry
EMOCIONES
EmergencyButton
EmergencyModal
EmotionSelector
EmotionStates

backend/src/services/observability.service.js

```
1.  /**
2.   * @file Servicio de observabilidad y monitoreo de la aplicación
3.   * @description Proporciona funcionalidades de logging, métricas Prometheus
4.   * @requires prom-client - Client de Prometheus para métricas
5.   * @requires winston - Logger estructurado
6.   * @requires winston-daily-rotate-file - Plugin para rotación de log
7.   */
8.
9.  const client = require('prom-client');
10. const winston = require('winston');
11. const DailyRotateFile = require('winston-daily-rotate-file');
12. const path = require('path');
13.
14. /**
15.  * Configuración de Prometheus Metrics
16.  */
17. const register = new client.Registry();
18.
19. // Métricas HTTP
20. const httpRequestDuration = new client.Histogram({
21.   name: 'http_request_duration_seconds',
22.   help: 'Duration of HTTP requests in seconds',
23.   labelNames: ['method', 'route', 'status_code'],
24.   registers: [register]
25. });
26.
27. const httpRequestCount = new client.Counter({
28.   name: 'http_requests_total',
29.   help: 'Total number of HTTP requests',
30.   labelNames: ['method', 'route', 'status_code'],
31.   registers: [register]
32. });
33.
34. const httpErrorCount = new client.Counter({
35.   name: 'http_errors_total',
36.   help: 'Total number of HTTP errors',
37.   labelNames: ['method', 'route', 'error_type'],
38.   registers: [register]
39. });
40.
41. // Métricas de autenticación
42. const authAttempts = new client.Counter({
43.   name: 'auth_attempts_total',
44.   help: 'Total authentication attempts',
45.   labelNames: ['endpoint', 'result'],
46.   registers: [register]
47. });
48.
49. // Métricas de base de datos
50. const mongoQueryDuration = new client.Histogram({
51.   name: 'mongo_query_duration_seconds',
52.   help: 'Duration of MongoDB queries in seconds',
```

```

53.     labelNames: ['operation', 'collection'],
54.     registers: [register],
55.     buckets: [0.001, 0.01, 0.1, 0.5, 1, 2, 5]
56. });
57.
58. const mongoQueryCount = new client.Counter({
59.     name: 'mongo_queries_total',
60.     help: 'Total number of MongoDB queries',
61.     labelNames: ['operation', 'collection', 'status'],
62.     registers: [register]
63. });
64.
65. /**
66.  * Configuración de Winston Logger
67.  */
68. const logsDir = path.join(__dirname, '..', 'logs');
69.
70. const logger = winston.createLogger({
71.     level: process.env.LOG_LEVEL || 'info',
72.     format: winston.format.combine(
73.         winston.format.timestamp({ format: 'YYYY-MM-DD HH:mm:ss' }),
74.         winston.format.errors({ stack: true }),
75.         winston.format.json()
76.     ),
77.     defaultMeta: { service: 'mindcare-api' },
78.     transports: [
79.         // Console output en desarrollo
80.         new winston.transports.Console({
81.             format: winston.format.combine(
82.                 winston.format.colorize(),
83.                 winston.format.printf(({ timestamp, level, message, ...meta }) => {
84.                     const metaStr = Object.keys(meta).length ? JSON.stringify(
85.                         meta, null, 2
86.                     ) : '';
87.                     return `${timestamp} [${level}]: ${message} ${metaStr}`;
88.                 })
89.             )
90.         });
91.
92.         // Agregar rotación de logs en producción
93.         if (process.env.NODE_ENV === 'production') {
94.             logger.add(new DailyRotateFile({
95.                 filename: path.join(logsDir, 'application-%DATE%.log'),
96.                 datePattern: 'YYYY-MM-DD',
97.                 maxSize: '20m',
98.                 maxDays: '14d',
99.                 format: winston.format.json()
100.            }));
101.
102.             logger.add(new DailyRotateFile({
103.                 filename: path.join(logsDir, 'error-%DATE%.log'),
104.                 level: 'error',
105.                 datePattern: 'YYYY-MM-DD',
106.                 maxSize: '20m',
107.                 maxDays: '30d',
108.                 format: winston.format.json()
109.            }));
110.        }
111.    ]
112. });
113.
114. /**
115.  * Middleware para registrar métricas HTTP
116.  * @param {Object} req - Objeto de request de Express
117.  * @param {Object} res - Objeto de response de Express
118.  * @param {Function} next - Función next de Express

```

```

117.  */
118.  const metricsMiddleware = (req, res, next) => {
119.      const start = Date.now();
120.
121.      // Patrones de rutas que se deben ignorar
122.      const ignorePaths = ['/api/metrics', '/api/health', '/health'];
123.      if (ignorePaths.includes(req.path)) {
124.          return next();
125.      }
126.
127.      // Interceptar el método send para capturar status code
128.      const originalSend = res.send;
129.      res.send = function(data) {
130.          const duration = (Date.now() - start) / 1000;
131.          const statusCode = res.statusCode;
132.          const route = req.route?.path || req.path;
133.          const method = req.method;
134.
135.          httpRequestDuration.observe(
136.              { method, route, status_code: statusCode },
137.              duration
138.          );
139.          httpRequestCount.inc({
140.              method,
141.              route,
142.              status_code: statusCode
143.          });
144.
145.          if (statusCode >= 400) {
146.              httpErrorCount.inc({
147.                  method,
148.                  route,
149.                  error_type: statusCode >= 500 ? 'server_error' : 'client_err
150.              });
151.          }
152.
153.          logger.info('HTTP Request', {
154.              method,
155.              path: req.path,
156.              statusCode,
157.              duration: `${duration.toFixed(3)}s`,
158.              ip: req.ip
159.          });
160.
161.          return originalSend.call(this, data);
162.      };
163.
164.      next();
165.  };
166.
167.  /**
168.   * Instrumentación para MongoDB con Mongoose
169.   * @param {Object} schema - Schema de Mongoose
170.   */
171.  const instrumentMongoDB = (schema) => {
172.      schema.pre(/.*/, function(next) {
173.          this._startTime = Date.now();
174.          next();
175.      });
176.
177.      schema.post(/.*/, function() {
178.          if (this._startTime) {
179.              const duration = (Date.now() - this._startTime) / 1000;
180.              const operation = this.op;

```

```
181.         const collection = this.model?.collection?.name || 'unknown';
182.
183.         mongoQueryDuration.observe(
184.             { operation, collection },
185.             duration
186.         );
187.
188.         mongoQueryCount.inc({
189.             operation,
190.             collection,
191.             status: 'success'
192.         });
193.
194.         logger.debug('MongoDB Query', {
195.             operation,
196.             collection,
197.             duration: `${duration.toFixed(3)}s`
198.         });
199.     }
200. });
201.
202. schema.post(/.*/, function(error, doc, next) {
203.     if (error) {
204.         const collection = this.model?.collection?.name || 'unknown';
205.         const operation = this.op;
206.
207.         mongoQueryCount.inc({
208.             operation,
209.             collection,
210.             status: 'error'
211.         });
212.
213.         logger.error('MongoDB Query Error', {
214.             operation,
215.             collection,
216.             error: error.message
217.         });
218.
219.         next(error);
220.     } else {
221.         next();
222.     }
223. });
224. };
225.
226. /**
227.  * Registrar intento de autenticación
228.  * @param {string} endpoint - Endpoint de autenticación
229.  * @param {string} result - Resultado ('success' o 'failure')
230.  */
231. const logAuthAttempt = (endpoint, result) => {
232.     authAttempts.inc({ endpoint, result });
233.     logger.info('Authentication Attempt', { endpoint, result });
234. };
235.
236. /**
237.  * Obtener métricas en formato Prometheus
238.  * @returns {Promise<string>} Métricas formateadas
239.  */
240. const getMetrics = async () => {
241.     return await register.metrics();
242. };
243.
244. module.exports = {
```

```
245.     logger,  
246.     metricsMiddleware,  
247.     instrumentMongoDB,  
248.     logAuthAttempt,  
249.     getMetrics,  
250.     register,  
251.     // Exportar métricas individuales para acceso directo  
252.     httpRequestDuration,  
253.     httpRequestCount,  
254.     httpErrorCount,  
255.     authAttempts,  
256.     mongoQueryDuration,  
257.     mongoQueryCount  
258.   };  
259.
```

Documentation generated by [JSDoc 4.0.5](#) on Fri Feb 13 2026 08:34:16 GMT+0000 (Coordinated Universal Time) using the [docdash](#) theme.